

Listas Enlazadas

Nombre del autor

 **Tiempo estimado: 30 min**

Universidad San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ingeniería en Ciencias y Sistemas.
Segundo Semestre 2026
[Laboratorio - Nombre del Curso - Sección]

1. Marco Formativo

1.1 Valores

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Ética	El estudiante asume la responsabilidad de comprender la lógica de punteros y la gestión dinámica de memoria antes de la sesión práctica. Es consciente de que una implementación deficiente puede derivar en fugas de memoria o corrupción de datos.

1.2 Competencia(s)

Tipo de Competencia	
Competencia General	Implementa estructuras de datos lineales y no lineales usando programación orientada a objetos para desarrollar soluciones eficientes.
Competencia Específica	Analiza y construye estructuras de datos no contiguas mediante la gestión dinámica de memoria, implementando la lógica de punteros para la vinculación de nodos y la resolución de punteros de autorreferencia, asegurando la integridad de la información durante la reestructuración de enlaces y optimizando el acceso secuencial en diferentes variantes de listas

2. Palabras Clave

- **Nodo:** La unidad básica de una lista enlazada que contiene tanto el valor del dato como uno o más punteros hacia otros nodos.
- **Puntero de Autorreferencia:** Un campo dentro de una estructura o clase que apunta a otra instancia del mismo tipo, permitiendo la creación de cadenas de datos.
- **Gestión Dinámica de Memoria:** El proceso de asignar y liberar espacio en el *Heap* durante la ejecución del programa, permitiendo que la estructura crezca o se encoja según sea necesario.
- **Encabezado (Head):** Puntero especial que marca el inicio de la lista y sirve como punto de entrada único para realizar cualquier operación de recorrido.
- **Asignación No Contigua:** Característica de las listas donde los elementos se almacenan en direcciones de memoria dispersas, a diferencia de los arreglos donde todos los elementos están uno tras otro.

- **Desreferenciación:** El acto de acceder al valor almacenado en la dirección de memoria a la que apunta un puntero, esencial para navegar entre los nodos de la lista.
- **Recorrido Secuencial:** Método de acceso donde, para llegar al elemento n , es obligatorio pasar por todos los nodos anteriores, dado que la ubicación del siguiente nodo solo se conoce estando en el actual.
- **Lista Doblemente Enlazada:** Variante de la estructura donde cada nodo posee dos punteros siguiente y anterior, permitiendo la navegación bidireccional y una mayor flexibilidad en la eliminación de nodos.

3. Introducción

A diferencia de las estructuras de almacenamiento contiguo, donde el tamaño y la ubicación de los datos están predeterminados, las listas enlazadas representan un salto hacia la eficiencia y la adaptabilidad en el desarrollo de software. Al basar su arquitectura en la gestión dinámica de memoria y el uso estratégico de punteros, estas estructuras permiten que la información se distribuya de forma orgánica en el sistema, creciendo o disminuyendo según las necesidades del entorno en tiempo real. Comprender su naturaleza no es solo aprender a organizar datos, sino dominar la lógica de la autorreferencia para construir sistemas capaces de gestionar recursos de manera óptima y fluida.

4. Contenidos

Presentación de temas y subtemas.

4.1 Estructuras de Datos Dinámicas y Gestión de Memoria

4.1.1 La Naturaleza del Almacenamiento No Contiguo

A diferencia de los arreglos tradicionales, donde la localización de un elemento se puede calcular mediante una fórmula matemática fija (mapeo lexicográfico), las listas enlazadas rompen la linealidad física de la memoria. En este paradigma, la relación entre los datos no depende de su cercanía en el hardware, sino de la lógica de punteros.

Cada elemento o nodo se convierte en una entidad autónoma que conoce su propia ubicación y la de su sucesor, permitiendo una flexibilidad total para insertar o eliminar información sin necesidad de desplazar grandes bloques de datos. Este enfoque es fundamental para optimizar sistemas que requieren un manejo constante de memoria en tiempo de ejecución.

Característica	Memoria Estática (Arreglos/Matrices)	Memoria Dinámica (Listas Enlazadas)
Asignación	En tiempo de compilación.	En tiempo de ejecución (Runtime).
Ubicación	Bloque contiguo (Memoria lineal).	Bloques dispersos (Heap/Montículo).
Acceso	Aleatorio y directo ($O(1)$).	Secuencial mediante punteros ($O(n)$).

Costo de Inserción	Alto (requiere mover elementos).	Bajo (solo reestructuración de enlaces).
Cálculo de Dirección	Fórmulas Row-Major / Column-Major.	Desreferenciación de punteros.
Gestión de Recursos	Automática por el sistema.	Manual (requiere liberar memoria).

Tabla 1 - Comparativa entre Memoria Estática y Memoria Dinámica

4.2 Estructuras de Datos Dinámicas y Gestión de Memoria

4.1.1 La Naturaleza del Almacenamiento No Contiguo

La manipulación de listas enlazadas no se basa en el desplazamiento físico de valores, sino en la reasignación de direcciones. Para realizar una operación exitosa, el programador debe asegurar que no se rompa el hilo conductor de la estructura; de lo contrario, se pierde el acceso al resto de la cadena, lo que resulta en una fuga de memoria.

El proceso crítico ocurre durante la inserción y eliminación, donde se deben seguir pasos estrictos: primero establecer el nuevo enlace y luego romper el anterior. Esta "coreografía" de punteros garantiza que la estructura mantenga su integridad incluso cuando el sistema operativo asigna los nuevos nodos en sectores de memoria totalmente aleatorios.

Operación	Descripción Técnica	Complejidad (Big O)	Impacto en Memoria
Inserción al Inicio	Actualización del puntero <i>Head</i> hacia	$O(1)$	Mínimo (creación de



	el nuevo nodo.		1 nodo).
Búsqueda de Valor	Recorrido lineal desde el inicio hasta hallar el nodo o NULL.	$O(n)$	Nulo (solo lectura).
Eliminación Intermedia	Bypass del nodo objetivo conectando el anterior con el siguiente.	$O(n)$	Positivo (liberación con free/delete).
Recorrido Completo	Visita sistemática de cada nodo para procesos de salida o conteo.	$O(n)$	Nulo (uso de puntero auxiliar).

Tabla 2 - Análisis de Complejidad y Casos de Uso en Operaciones de Lista

5. Conclusiones

Versatilidad frente a la Memoria Estática: A diferencia de las matrices que dependen de fórmulas de mapeo rígidas, las listas enlazadas ofrecen una solución superior para problemas donde el volumen de datos es impredecible, permitiendo un crecimiento dinámico que optimiza el uso del *Heap*.

Importancia Crítica de la Lógica de Punteros: La integridad de una lista no reside en los datos que contiene, sino en la correcta gestión de sus enlaces. Un error en la asignación de punteros no solo corrompe la estructura, sino que puede provocar fallos críticos del sistema como desbordamientos o fugas de memoria.

Eficiencia en Operaciones de Reestructuración: Las listas enlazadas superan a los arreglos en operaciones de inserción y eliminación, ya que eliminan la necesidad de desplazar físicamente los elementos en memoria, limitándose a una simple actualización de direcciones lógicas.

Costo de Acceso y Recorrido: Se concluye que la flexibilidad de las listas tiene un costo en términos de tiempo de acceso. Al perder la capacidad de acceso aleatorio ($O(1)$) de las estructuras lineales, el programador debe aceptar una complejidad de búsqueda lineal ($O(n)$).

Adaptabilidad Arquitectónica: La naturaleza de los nodos y punteros de autorreferencia permite que la estructura evolucione fácilmente hacia formas más complejas (dobles, circulares o incluso árboles y grafos), convirtiéndolas en la unidad fundamental para construir arquitecturas de datos avanzadas.

6. Referencias

1. **Sedgewick, R., & Wayne, K. (2011).** Algorithms (4th ed.). Addison-Wesley Professional.
<https://algs4.cs.princeton.edu/13stacks/>
2. **Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022).** Introduction to Algorithms (4th ed.). MIT Press.
<https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/>